

From Root Objects to Radicals

An exact Galois–Kummer reference solver
with structural fast paths and explicit branch selection

Prepared for Vladimir Reshetnikov

8 September 2026 | Reference release 0.1.0

Abstract

An algebraic number represented by a polynomial `Root` can admit a radical expression even when `ToRadicals` leaves it unchanged. The obstruction is not polynomial degree alone: the exact criterion is solvability of the Galois group of the minimal polynomial. We turn this criterion into a finite constructive algorithm. It computes an embedded normal number field, adjoins only the prime roots of unity required by the group order, builds a prime-cyclic subgroup chain, and uses trace–Fourier resolvents to construct Kummer generators. Rational linear algebra expresses every radicand in the preceding field; exact embedded-number comparisons select principal-power branches. The resulting expression contains no residual `Root`, trigonometric, or exponential placeholders.

The accompanying Wolfram Language and Python sources also implement less expensive structural reductions: power composition, Dickson polynomials, generalized reciprocal polynomials, pair-sum resolvents, and supplied radical extensions. The two examples in the motivating question receive especially short solutions. The normal-field algorithm is mathematically complete with unbounded exact arithmetic; the delivered software is a bounded reference implementation, not a claim of practical success on every degree or every backend input. Python execution tests and Wolfram tests supplied for local execution are distinguished explicitly.

Release contract. Exact parameter-free algebraic numbers over $\mathbb{Q}$; principal complex powers; explicit success, nonsolvability, resource-limit, and inconclusive outcomes. Radical depth and expression length are not minimized.

Contents

1	The problem and the deliverable	3
1.1	What “any <code>Root</code> ” means here	3
1.2	Output syntax and branch semantics	3
2	What the supplied notebook contributes	4
3	Solvability and the normal closure	5
4	An exact representation of the normal field	5
4.1	Computing automorphisms without a Galois-group oracle	6
4.2	A Wolfram-specific primitive-element requirement	6

5	A small sufficient cyclotomic base	6
6	The finite subgroup calculation	7
7	A deterministic Kummer generator	7
8	From abstract roots to principal powers	8
9	Coordinates, a radical tower, and output	9
9.1	Straight-line output versus full expansion	10
10	Structural fast paths	10
10.1	Low degree and affine power composition	10
10.2	Dickson polynomials and correlated radicals	10
10.3	Generalized reciprocal reduction	11
10.4	Pair-sum resolvents	11
10.5	Supplied radical extensions	12
11	Worked examples	12
11.1	The quintic from the question	12
11.2	The sextic becomes a cubic and a quadratic	12
11.3	A degree-eight notebook example	13
11.4	Forcing the general method	13
12	Using the software	14
12.1	Wolfram Language	14
12.2	Python API and command line	14
12.3	Result categories	15
13	Validation and its limits	16
14	Cost, optimization, and what is not implemented	16
15	Conclusion	17
A	Reproduction and source map	17
B	A schematic algorithm	18

1 The problem and the deliverable

The motivating question asks why certain algebraic numbers remain as `Root` objects despite possessing radical expressions [1]. Wolfram’s documentation expressly distinguishes its degree-four guarantee from higher-degree cases that can be missed [4]. A more systematic solution should therefore do more than apply extra simplification rules.

There are three separate tasks. First, determine whether a radical expression exists. Second, construct one when it does. Third, ensure that the expression represents the *selected complex embedding*, not merely an unspecified root of the same polynomial. This package addresses all three at the algorithmic level, while exposing resource and backend limitations instead of confusing them with a mathematical obstruction.

The archive contains a native Wolfram Language package, a separately usable Python implementation built on SymPy, test suites, examples, an inspection-only extraction of the supplied notebook’s input cells, and this article. The general backend is implemented in both languages; it is not an instruction to call an unimplemented “solve the Galois problem” routine. It constructs automorphisms and Kummer generators using explicit field operations. The Python implementation has been executed in the preparation environment. The Wolfram evaluation service returned HTTP 404, so its package has not been executed in a Wolfram kernel here. The supplied `.wlt` tests make that missing validation reproducible locally.

1.1 What “any Root” means here

An input is a fixed $\alpha \in \overline{\mathbb{Q}} \subset \mathbb{C}$, represented with an exact embedding. The Wolfram interface accepts an exact algebraic expression, including a polynomial `Root`; the Python interface accepts a univariate rational polynomial and a SymPy root index. Parameters, floating-point coefficients, transcendental input, and inverse-function `Root` objects that are not algebraic numbers over $\mathbb{Q}$ are outside this release’s contract.

A reducible defining polynomial must first be reduced to the irreducible factor containing the selected root. For example, a rational root of $(x - 7)(x^5 - x - 1)$ is certainly radical-expressible even though the other factor has a nonsolvable Galois group. The Python interface uses the selected `CRootOf`’s factor; the Wolfram interface computes a minimal polynomial.

1.2 Output syntax and branch semantics

Definition 1.1. A radical expression is generated by rational constants, i , addition, multiplication, and rational powers, with all subexpressions well-defined. Negative integer powers include division. Each noninteger power is the principal complex power. The implementation’s grammar test rejects algebraic-root objects, free symbols, floating-point literals, and trigonometric or exponential functions.

Thus $\cos(2\pi/7)$ is not accepted as an answer merely because it is algebraic. A primitive root of unity, however, has the allowed representation

$$z_n = (-1)^{2/n} \quad (n > 1), \quad z_1 = 1. \quad (1)$$

Since the principal argument of -1 is π , this is precisely $\exp(2\pi i/n)$. Equation (1) is used as a radical, not as an unevaluated exponential. It may not be a shortest radical expression for the same constant. Real targets are allowed to have complex intermediate values; requiring every intermediate radical to remain real is a different problem.

2 What the supplied notebook contributes

The supplied archive contains one notebook, `FindExtension.nb`, of 986,737 bytes. A non-evaluating box reader extracted 220 input cells for inspection. The extraction is deliberately not advertised as executable source: certain special boxes and incomplete scratch cells retain display-oriented syntax. The original notebook was read, not executed. Source locations below refer to line numbers in its textual notebook representation, not input-cell numbers.

The experiments contain two valuable ideas: search for coefficients of low-degree factors, and use combinations of conjugates to discover smaller extension fields. Those are recognizable fragments of resolvent and fixed-field methods. A systematic package should retain these ideas while replacing unbounded search, implicit branch choices, and state-dependent notebook execution.

Notebook nent	compo-	Interpretation and release response
<code>findExtension</code> , line 65		Searches for a coefficient of a quadratic factor by elimination. A pair-sum resolvent gives a direct exact polynomial for such coefficients. A small coefficient field is a useful candidate, not by itself a proof that the chosen target has been solved.
<code>findExtension3</code> , line 126		Analogous cubic-factor experiments suggest higher subset resolvents. The release implements pair sums only; arbitrary subset resolvents are discussed as a future fast-path improvement.
<code>project</code> , line 182		Real and imaginary parts of conjugates may have smaller minimal polynomials. A successful projection still needs a target reconstruction step; failure says nothing about solvability.
<code>factor</code> , line 252; <code>solve</code> , line 315		Candidate selection uses <code>PossibleZeroQ</code> , or the first real solution. The release uses exact embedded-number equality and explicit failure outcomes. A “first real” rule cannot generally preserve an arbitrary input root.
<code>fuzz</code> , line 436		Random subsets, 400-digit approximation and <code>RootApproximant</code> are useful exploratory tools. The displayed infinite loop has no exhaustion exit even after its finite subset space is exhausted. No finite failure criterion or complete search schedule is supplied. The general solver instead uses a finite trace-basis search.
<code>radicalDepth</code> , line 21		This is a syntactic score, not a solvability test. In particular a residual <code>Root</code> can fall into its depth-zero case. The new grammar test excludes residual roots before accepting an answer.
<code>simplify</code> , line 560		Randomized transformations, an expanding time budget, and private simplification scores are separated from construction in the release. The earlier selectors spell <code>Simplify'SimpifyCount</code> , whereas this cell uses <code>Simplify'SimplifyCount</code> ; the misspelling should not be relied on as a complexity metric. Neither private name is used in the package.

For a monic quadratic divisor $x^2 + px + q$, remainder vanishing is equivalently vanishing of its two coefficient expressions. This is simpler to specify and audit than negating an elimination of a nonvanishing remainder. Moreover, $p = -(r_i + r_j)$ for two roots. A repeated pair sum can correspond to different unordered pairs, so even knowing p need not determine q uniquely. Such collisions must not be erased by careless square-free processing.

There are also practical notebook concerns: unrestricted use of global x , empty selections

followed by [[1]], cells depending on earlier interactive state, and expression-size growth. These are reasons to use explicit arguments, lexically scoped variables, and structured result records. They do not invalidate the successful identities found experimentally.

The notebook's large degree-nine, degree-eighteen, and degree-thirty experiments were not all converted into regression fixtures or benchmarked here. The release includes one explicit degree-eight example from it in addition to the motivating question. No claim of complete notebook-corpus coverage is made.

3 Solvability and the normal closure

Let $f \in \mathbb{Q}[x]$ be the monic irreducible minimal polynomial of α , let L be its splitting field inside our fixed copy of $\mathbb{C}$, and put

$$G = \text{Gal}(L/\mathbb{Q}), \quad d_L = [L : \mathbb{Q}] = |G|.$$

The classical solvability criterion and the cyclic-extension theorem are the starting points, not new results of this release [2, Prop. 5.27, Thm. 5.34]. The construction below spells out the finite searches and coordinate operations needed to turn them into code.

Theorem 3.1. *The selected α has a radical expression over $\mathbb{Q}$ if and only if G is solvable. For an irreducible f , either every conjugate is radical-expressible or none is.*

Proof. A radical tower containing α can be enlarged by roots of unity and the conjugates of its generators so that its normal closure has a solvable Galois group. One can see this by adjoining the conjugates layer by layer: after the preceding layer and the needed roots of unity have been included, the new layer is generated by roots of binomials and contributes an abelian kernel. The splitting field of f is a normal subfield of this enlarged field, so its Galois group is a quotient of a solvable group. This proves necessity for one selected root, not merely for a prior promise about all roots.

Conversely, Sections 5–9 construct a radical tower containing L when G is solvable and give an expression for the specific embedded α . Applying the same final coordinate step to any conjugate gives the last assertion. $\square$

An extension $\mathbb{Q}(\alpha)/\mathbb{Q}$ need not be normal even when α is easily expressible by radicals. Its automorphism group alone is therefore not the correct obstruction. For instance, $\mathbb{Q}(\sqrt[3]{2})$ has too few internal automorphisms to encode the full cubic splitting problem. The normal closure is essential to the completeness argument.

4 An exact representation of the normal field

All roots of f can be represented as exact algebraic numbers without first finding radicals. A primitive-element operation on the list gives $L = \mathbb{Q}(\theta_L)$. This is not circular: exact algebraic-root isolation, field membership, and factoring in number fields are permitted backend operations; radical extraction for arbitrary algebraic numbers is not assumed.

For an embedded field $M = \mathbb{Q}(\theta)$ with minimal polynomial P of degree D , represent an element by its unique polynomial of degree less than D in $\mathbb{Q}[X]/(P)$. Addition and multiplication are rational polynomial operations followed by reduction modulo P . An inverse is computed by the Euclidean algorithm. Equality in this representation is exact equality of coefficient vectors. The fixed complex embedding is carried by the choice of θ . The CAS representations used here are documented in [6, 8].

4.1 Computing automorphisms without a Galois-group oracle

If $M/\mathbb{Q}$ is normal, factor P over M . It must have D distinct linear factors. Write their roots as $t_g(\theta)$ with $\deg t_g < D$. Then

$$g(h(\theta)) = h(t_g(\theta)), \quad h \in \mathbb{Q}[X]/(P). \quad (2)$$

Every embedding is an automorphism, and every automorphism appears once. A factorization failing the linearity or root-count check is a backend failure, not a reason to silently continue.

Composition deserves care. With the convention $(g \circ h)(a) = g(h(a))$, the image polynomial for $g \circ h$ is

$$t_h(t_g(X)) \pmod{P(X)}.$$

The apparent reversed order is intentional. The group table uses exactly this convention. Explicit substitution makes the table associative; the Python general-purpose finite-group test helper also checks associativity when given an external table.

4.2 A Wolfram-specific primitive-element requirement

The call `ToNumberField[list, All]` is used, not its default second argument. The documentation says that `All` chooses the smallest common extension, whereas the automatic form need not do so [5]. An unrelated larger field could invalidate the intended normal-field or degree arguments and needlessly inflate the computation.

The package also makes the chosen generator integral. If $a_n X^n + \dots + a_0$ is its primitive integer minimal polynomial, then $a_n \theta$ is integral and generates the same field. The documentation promises representation in the specified generator when that generator is integral. The encoder checks that it has not silently changed the generator before using `AlgebraicNumberPolynomial`. This avoids interpreting coordinate coefficients relative to the wrong primitive element.

5 A small sufficient cyclotomic base

Set

$$m = \text{rad}(d_L) = \prod_{p|d_L} p, \quad K = \mathbb{Q}(z_m), \quad M = LK. \quad (3)$$

Only distinct prime divisors are needed: the subgroup chain below has *prime* cyclic quotients, not arbitrary prime-power cyclic quotients. The value $m = 1$ is used for $d_L = 1$. This is a sufficient conductor, not a claim of a minimal conductor. For example the factor 2 can sometimes be omitted without changing the cyclotomic field.

Both L and K are normal over $\mathbb{Q}$, hence so is M . Let

$$H = \text{Gal}(M/K), \quad D = [M : \mathbb{Q}], \quad d_0 = [K : \mathbb{Q}] = \varphi(m).$$

Starting with all automorphisms from (2), retain exactly those fixing the stored field element z_m . This gives H , and the implementation checks

$$|H|\varphi(m) = D. \quad (4)$$

Proposition 5.1. *The groups H and G are solvable simultaneously. Every prime dividing $|H|$ divides m .*

Proof. Restriction identifies H with $\text{Gal}(L/L \cap K)$, a normal subgroup of G . The field $L \cap K$ is a Galois subfield of the abelian extension $K/\mathbb{Q}$; therefore $G/\text{Gal}(L/L \cap K)$ is abelian. A subgroup of a solvable group is solvable, and an extension of a solvable group by an abelian group is solvable. Thus the equivalence follows. Finally $|H|$ divides $|G| = d_L$, so every prime factor of $|H|$ occurs in the product defining m . $\square$

This observation prevents a common incorrect shortcut: adjoining roots of unity cannot turn a genuinely nonsolvable polynomial over $\mathbb{Q}$ into a solvable one while hiding the obstruction in an unexamined quotient. Here the quotient is explicitly known to be abelian.

6 The finite subgroup calculation

Compute the derived series

$$H^{(0)} = H, \quad H^{(j+1)} = [H^{(j)}, H^{(j)}].$$

Because the groups are finite, the orders either decrease or stabilize. A nontrivial stable derived subgroup certifies nonsolvability. This is the only kind of conclusion called `NotSolvable` by the generic group path.

For a solvable current subgroup S , construct a maximal proper subgroup containing its commutator subgroup as follows. Start with $N = [S, S]$. Scan the elements $g \in S$ in fixed order. If $\langle N, g \rangle$ is still proper in S , replace N by that subgroup; otherwise retain N . The scan needs no backtracking. Once $\langle N, g \rangle = S$, increasing N cannot make the same join proper again.

Lemma 6.1. *At the end of the scan, $N \triangleleft S$ and S/N is cyclic of prime order.*

Proof. Every intermediate subgroup contains $[S, S]$ and is therefore the inverse image of a subgroup of the abelian group $S/[S, S]$. In particular it is normal in S . The preceding monotonicity observation proves maximality after one scan. The quotient S/N is a nontrivial simple abelian group, hence cyclic of prime order. $\square$

Repeat with $S = N$ until the identity subgroup is reached. This produces

$$H = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_r = \{1\}, \quad [H_{i-1} : H_i] = p_i \text{ prime.} \quad (5)$$

Each subgroup only has to be normal in the immediately preceding subgroup, not in all of H . Explicit normality and prime-index checks are retained in the code even though the construction proves them. The corresponding fixed fields satisfy

$$K = M^{H_0} \subset M^{H_1} \subset \cdots \subset M^{H_r} = M,$$

with each adjacent extension cyclic of degree p_i .

7 A deterministic Kummer generator

Fix one step $S = H_{i-1}$, $N = H_i$, $p = [S : N]$, and choose $\sigma \in S \setminus N$. Its restriction generates $\text{Gal}(M^N/M^S)$. Let $\zeta = z_m^{m/p}$, a primitive p th root of unity lying in K . For $k = 0, \dots, D-1$, form

$$a_k = \sum_{\nu \in N} \nu(\theta^k), \quad (6)$$

$$\rho_k = \sum_{j=0}^{p-1} \zeta^{-j} \sigma^j(a_k). \quad (7)$$

Choose the first nonzero ρ_k by exact field arithmetic. No random coefficient bounds or guessed algebraic approximants are involved.

Lemma 7.1 (Finite nonvanishing search). *At least one of the D values in (7) is nonzero.*

Proof. The elements $1, \theta, \dots, \theta^{D-1}$ span M over $\mathbb{Q}$. The map $T_N : b \mapsto \sum_{\nu \in N} \nu(b)$ is a $\mathbb{Q}$ -linear surjection onto M^N : for $b \in M^N$, $T_N(b/|N|) = b$. Consequently the a_k span M^N over $\mathbb{Q}$.

The distinct automorphisms $1, \sigma, \dots, \sigma^{p-1}$ of M^N/M^S are linearly independent as maps over M^N (independence of field homomorphisms). Therefore the map $A = \sum_j \zeta^{-j} \sigma^j$ is not the zero map. If every $A(a_k)$ vanished, $\mathbb{Q}$ -linearity and the spanning property would force $A = 0$, a contradiction. $\square$

Proposition 7.2 (Constructive cyclic radical step). *For the chosen $\rho = \rho_k \neq 0$,*

$$\nu(\rho) = \rho \ (\nu \in N), \quad \sigma(\rho) = \zeta \rho, \quad c = \rho^p \in M^S,$$

and $M^N = M^S(\rho)$.

Proof. Normality of N in S implies that σ preserves M^N , and each a_k belongs to M^N . Thus so does ρ . Reindexing the Fourier sum gives $\sigma(\rho) = \zeta \rho$, using $\sigma^p(a_k) = a_k$ and $\sigma(\zeta) = \zeta$. Taking p th powers shows that c is fixed by σ and by N , hence by S . Since $\zeta \neq 1$ and $\rho \neq 0$, ρ is not fixed by σ and does not belong to M^S . The intermediate degree divides the prime p and is not one; it is therefore p , proving the field-generation claim. $\square$

The code rechecks these eigenvector and fixed-field identities modulo P . This finite basis search is the principal replacement for the notebook's unbounded randomized searches. It does not guarantee a small generator; it guarantees a generator once the exact field and group data are available.

8 From abstract roots to principal powers

A relation $\rho^p = c$ by itself is insufficient for an explicit answer using principal powers. It identifies p choices, not the chosen embedded ρ . Let $q = c^{1/p}$ be the principal root of the exact embedded value c . Since $q^p = \rho^p$ and $c \neq 0$,

$$\rho = \zeta^b q$$

for exactly one $b \in \{0, \dots, p-1\}$. All these candidates lie in M . The implementation finds b using exact embedded field membership and equality. It does not accept a small floating-point residual as proof.

The Wolfram version computes `RootReduce` on each difference and requires literal zero. The Python version re-embeds the principal power in its exact algebraic field and compares the reduced coordinate arrays. These checks trust the respective CAS number-field and embedding machinery. They are not a formal proof object verified independently of that machinery. Arbitrary precision may be used internally by a CAS to identify embeddings, but no user-level tolerance test is substituted for an algebraic equality.

Remark 8.1 (A useful failure test). The two numbers $\sqrt{2}$ and $-\sqrt{2}$ have the same minimal polynomial. Consequently checking only $f(e) = 0$, or equality of minimal polynomials, would accept a wrong conjugate. Both test suites explicitly reject this case.

No `PowerExpand` operation is used. In particular, the generally false principal-power identity $(uv)^{1/p} = u^{1/p}v^{1/p}$ is never needed. The only branch adjustment is the finite verified multiplier ζ^b .

9 Coordinates, a radical tower, and output

Maintain two parallel objects: a rational basis B_i of M^{H_i} as exact elements of M , and a symbolic basis $\mathcal{B}_i$ giving their already constructed radical expressions. Initially

$$B_0 = (1, z_m, \dots, z_m^{d_0-1}), \quad \mathcal{B}_0 = (1, z_0, \dots, z_0^{d_0-1}),$$

where the first straight-line definition is $z_0 = (-1)^{2/m}$ (or $z_0 = 1$).

At the i th step, write the exact element $c_i = \rho_i^{p_i}$ in the preceding basis:

$$c_i = \sum_{b \in B_{i-1}} q_{i,b} b, \quad q_{i,b} \in \mathbb{Q}. \quad (8)$$

The system is overdetermined in ambient coordinates but has a unique solution: the basis is independent and c_i belongs to the lower field. A rational linear solver computes these coefficients; the implementation checks the reconstructed vector exactly. Define

$$r_i = z_0^{b_i m / p_i} \left(\sum_{b \in B_{i-1}} q_{i,b} \mathcal{B}_{i-1}(b) \right)^{1/p_i}. \quad (9)$$

By the branch check, its embedded value is exactly ρ_i . Update the bases to the ordered lists

$$B_i = (b \rho_i^j)_{0 \leq j < p_i, b \in B_{i-1}}, \quad \mathcal{B}_i = (\mathcal{B}_{i-1}(b) r_i^j)_{0 \leq j < p_i, b \in B_{i-1}}. \quad (10)$$

The product-basis theorem and Proposition 7.2 prove independence. The size check is

$$|B_i| = \varphi(m) \prod_{j=1}^i p_j = D/|H_i|.$$

At the end B_r is a rational basis of M . Solve one more coordinate system for α , then substitute the definitions of $z_0, r_1, \dots, r_r$ in order.

Theorem 9.1 (Constructive correctness and termination). *Given terminating exact algorithms for rational polynomial factorization, embedded algebraic-number arithmetic, primitive elements, and factorization over number fields, the unbounded normal-field procedure terminates. It either proves nonsolvability or returns a radical expression equal to the selected α . It returns a radical expression precisely when one exists.*

Proof. The initial splitting field and cyclotomic compositum have finite degree, and the number-field operations have the stated terminating contracts. The group and each subgroup scan are finite. If the derived series stabilizes nontrivially, Proposition 5.1 and Theorem 3.1 prove nonsolvability. Otherwise all subgroup indices are prime and divide m . Lemma 7.1 makes each resolvent search finite, and Proposition 7.2 establishes the required cyclic radical steps.

Induction on (10), using the exact branch equality at each step, proves that every symbolic basis expression has the value of its paired field element. Equation (8) always has the claimed unique rational solution. At the final step the coordinate expression therefore equals α . Its definitions use only allowed arithmetic and rational powers, with nonzero Kummer radicands. This proves correctness and existence of all required steps. The solvability criterion proves the converse. $\square$

The conditional backend clause is important. A mathematical decision algorithm is not a guarantee that a particular version of a CAS will successfully finish every call before memory exhaustion, a time limit, or an implementation exception. The delivered wrappers expose those outcomes. Their default bounded settings are *not* a complete decision procedure in finite practical time.

9.1 Straight-line output versus full expansion

A nested radical can contain the same large subexpression repeatedly. The result records retain symbolic definitions and a final value expression so that a consumer can inspect a straight-line program instead of reading one enormous expanded formula. The normal-field implementations also construct an expression with all temporary symbols substituted. This can itself be expensive and there is no expression-size minimizer in this release.

The certificates contain chain indices, branches, radicands, eigenvector coordinates and a group multiplication table. They are audit records, not a standalone portable formal certificate format: an independently trusted verifier would additionally need a complete serialized embedded-field presentation and embedding certificates. Python's `verify_result` provides an independent final CAS equality check; the Wolfram routine always performs its final `RootReduce` equality check before reporting success.

10 Structural fast paths

A normal closure is often unnecessary for a short answer. The solver first tries exact structural transformations whose candidate results are verified against the selected root. Exhausting them yields “not found,” never “not solvable.”

10.1 Low degree and affine power composition

The usual degree-at-most-four formulas supply initial candidates. A backend may leave a formula incomplete or retain a root object; the grammar and equality checks prevent such an object from being counted as successful radical output.

For monic f of degree n , let $h = -[x^{n-1}]f/n$, so that $f(x+h)$ is centered. Take the greatest common divisor d of the positive exponents with nonzero coefficients. If $d > 1$, then

$$f(x+h) = g(x^d), \quad \deg g = n/d.$$

Solve $g(y) = 0$, and for each candidate β try

$$h + z_d^k \beta^{1/d}, \quad 0 \leq k < d.$$

Recursion strictly reduces polynomial degree. The fast-path recursion has an additional safety depth limit in this release, so it remains a bounded candidate generator, not the source of the completeness theorem.

10.2 Dickson polynomials and correlated radicals

Define

$$D_0(x, a) = 2, \quad D_1(x, a) = x, \quad D_n(x, a) = xD_{n-1}(x, a) - aD_{n-2}(x, a).$$

Induction gives the central identity

$$D_n(u + a/u, a) = u^n + (a/u)^n \quad (u \neq 0). \quad (11)$$

If a centered polynomial is exactly $D_n(x, a) + c$, solve

$$U^2 + cU + a^n = 0.$$

For a nonzero solution U and $u = z_n^k U^{1/n}$, equation (11) gives the candidate $x = u + a/u$. A preceding affine centering shift is added back. The coefficient of x^{n-2} is $-na$, providing an

immediate candidate for a ; the entire polynomial identity is checked, not inferred from one coefficient. Even degrees are already eligible for the power-composition reduction.

The dependent term a/u is crucial. Independently selecting an n th root of U and an n th root of a^n/U need not produce a pair whose product is a . The correlated form automatically preserves that constraint. This is the systematic version of the substitution $x = y + 1/y$ discussed in the original question's answers [1].

10.3 Generalized reciprocal reduction

Let $f(x) = \sum_{k=0}^{2q} c_k x^k$ be monic and suppose a nonzero rational a satisfies

$$c_{q-j} = a^j c_{q+j} \quad (1 \leq j \leq q). \quad (12)$$

Then

$$f(x) = x^q g(x + a/x), \quad g(t) = c_q + \sum_{j=1}^q c_{q+j} D_j(t, a). \quad (13)$$

Indeed division by x^q pairs the j th positive and negative powers, and $D_j(x + a/x, a) = x^j + a^j x^{-j}$ by (11). Thus a degree- $2q$ equation reduces to a degree- q equation followed by

$$x = \frac{t \pm \sqrt{t^2 - 4a}}{2}.$$

The nonzero constant coefficient excludes the lost value $x = 0$. The first nonzero upper coefficient in (12) yields finitely many rational candidates for a ; all coefficient relations are then checked exactly. The familiar reciprocal and anti-reciprocal-looking cases are only special instances of this identity.

10.4 Pair-sum resolvents

Let $r_1, \dots, r_n$ be the distinct roots of monic f . Define

$$R_2(y) = \prod_{i < j} (y - r_i - r_j) \in \mathbb{Q}[y].$$

A direct resultant identity is

$$\text{Res}_x(f(x), f(y-x)) = \prod_{i,j} (y - r_i - r_j) = 2^n f(y/2) R_2(y)^2. \quad (14)$$

This gives an exact implementation: divide out the diagonal factor, factor over $\mathbb{Q}$, and halve the exponents. Repeated pair sums remain repeated in R_2 ; taking only a square-free part would be wrong. The release checks the degree $n(n-1)/2$ and the exact division. The Python tests verify the full identity for the motivating sextic.

If a low-degree factor of R_2 gives a radical β , factor f over $\mathbb{Q}(\beta)$ and solve the smaller factors. The discovered β is an extension candidate; no claim is made that every such factor helps the selected root. The Python pair path restricts lifts to degrees at most three, and the Wolfram pair path uses the same restriction, to avoid very large quartic expressions. User-supplied extensions allow degree-four lifts. Pair resolvents are attempted only up to a configured original degree (default eight).

General subset sums or elementary symmetric functions would extend this search, but the degree $\binom{n}{k}$ and collisions can become expensive. They are not necessary for the complete normal-field backend and are not implemented here.

10.5 Supplied radical extensions

A caller may supply a known radical element generating a useful field, or in Wolfram a list of such generators. The package factors over that extension, lifts small factors, and verifies the target. For Python, a primitive element such as $\sqrt{3} + \sqrt{5}$ is convenient. This does not assert that *every* sum of suggested generators is primitive; a true compositum can instead be constructed explicitly before choosing a generator.

The extension must itself pass the radical grammar. Supplying an unsolved `Root` as a coefficient field would otherwise merely move the original problem into the coefficients and falsely present that as a radical solution.

11 Worked examples

11.1 The quintic from the question

The polynomial is

$$f_5(x) = 5x^5 - 25x^3 + 25x + 6 = 5(D_5(x, 1) + 6/5).$$

Let

$$u = \left(\frac{-3 + 4i}{5} \right)^{1/5}. \quad (15)$$

Then $u^5 + u^{-5} = -6/5$, hence all five candidates

$$x_k = z_5^k u + (z_5^k u)^{-1}, \quad 0 \leq k < 5,$$

satisfy $f_5(x) = 0$. Because $|u| = 1$, each is real. Put $\theta = \text{Arg}((-3 + 4i)/5)/5 \in (\pi/10, \pi/5)$. The five angles are $\theta + 2k\pi/5$; elementary comparison of their cosines shows that $k = 0$ gives the largest real root. Consequently the target

```
Root[6 + 25 # - 25 #^3 + 5 #^5 &, 5]
```

is represented by

$$\boxed{u + u^{-1}}. \quad (16)$$

The trigonometric observation is only used here to explain root order; the package output (16) contains no trigonometric expression. The Python suite independently verifies all five selected roots, not just this one.

11.2 The sextic becomes a cubic and a quadratic

For

$$f_6(x) = x^6 + x^4 - x^3 - x^2 - 1$$

we have the exact identity

$$f_6(x) = x^3 \left[(x - 1/x)^3 + 4(x - 1/x) - 1 \right]. \quad (17)$$

Thus take $t = x - 1/x$, solve $t^3 + 4t - 1 = 0$, then solve $x^2 - tx - 1 = 0$. A branch-correlated real Cardano expression is

$$C = \left(\frac{1}{2} + \frac{\sqrt{849}}{18} \right)^{1/3}, \quad t = C - \frac{4}{3C}. \quad (18)$$

Here $C > 0$. Writing $v = -4/(3C)$ gives $Cv = -4/3$ and $C^3 + v^3 = 1$, which proves $t^3 + 4t = 1$. Its derivative $3t^2 + 4$ is positive, so this is the cubic's unique real root. The two real sextic roots are

$$\frac{t - \sqrt{t^2 + 4}}{2} < 0 < \frac{t + \sqrt{t^2 + 4}}{2}.$$

The target `Root[... , 2]` for this sextic is therefore

$$\boxed{\frac{t + \sqrt{t^2 + 4}}{2}}. \quad (19)$$

The package may print a different but exactly equivalent cubic radical form. The test criterion is equality, not text identity with (18).

The same structure appears in the pair-sum resolvent:

$$\begin{aligned} R_2(y) &= (y^3 + 4y - 1)h(y), \\ h(y) &= y^{12} - y^9 + 8y^8 - 2y^7 - y^6 + 4y^5 \\ &\quad + 18y^4 + 9y^3 + 2y^2 - 1. \end{aligned}$$

Each paired quadratic has root sum t . This explains why a low-degree extension search can discover the cubic even when a direct sextic radical conversion does not.

11.3 A degree-eight notebook example

One explicit fixture from the notebook is

$$\begin{aligned} f_8(x) &= x^8 + 8x^7 + 20x^6 + 8x^5 - 36x^4 \\ &\quad - 48x^3 - 15x^2 + 2x + 1. \end{aligned}$$

Put $s = 2x + 2$. Its equation becomes

$$s^8 - 32s^6 + 224s^4 - 448s^2 + 256 = 0.$$

This identity can also be obtained by eliminating a, b from $a^2 = 3$, $(b^2 - 5)^2 = 20$, and $s = a + b$; the coefficient normalization is verified in `examples/identities.py` rather than assumed from numerical matching. Its smallest real root has the radical form

$$\frac{-2 - \sqrt{3} - \sqrt{5 + 2\sqrt{5}}}{2}. \quad (20)$$

The eight conjugates result from independent choices of the signs of $\sqrt{3}$, $\sqrt{5}$, and the outer square root. They are distinct, and all radicands $5 \pm 2\sqrt{5}$ are positive. Choosing both outer contributions negative and the larger radicand gives the smallest value. The Python regression uses the hint $\sqrt{3} + \sqrt{5}$, factors exactly over that field, and checks both its selected-root identity and equality with (20). This is a *supplied-extension* success, not a claim that the generic normal-field route was benchmarked on this degree-eight field.

11.4 Forcing the general method

A useful control experiment is $f = x^3 - 2$. Its splitting field has degree six. After adjoining the roots of unity specified by $m = 6$, the relative group over $\mathbb{Q}(z_6)$ has order three. The general backend constructs a single prime-index Kummer step. For the real root it returns an expression equal to $2^{1/3}$; a separate test targets a nonreal conjugate.

For $x^4 - 2$, the generic test builds the degree-eight splitting field. Since $m = 2$, the cyclotomic base is just $\mathbb{Q}$, and the observed chain has three quadratic steps. The final expression is exact but much less attractive than $-2^{1/4}$ for the smallest real root. This is expected: the purpose of this test is to exercise the normal-field construction, not to compete with the power-composition fast path.

12 Using the software

12.1 Wolfram Language

From the extracted archive directory:

```
Get["wolfram/RadicalRoot.wl"];
a = Root[6 + 25 # - 25 #^3 + 5 #^5 &, 5];
e = RootToRadicals[a, Method -> "Fast"];
VerifyRadical[a, e]
```

The expected verification result is `True`; this usage is supplied for local Wolfram execution, not reported as executed here. To retain construction data and force the general backend:

```
r = RadicalSolve[Root[#^3 - 2 &, 1],
  Method -> "Galois", TimeConstraint -> 300];
If[AssociationQ[r], r["Certificate"], r]
```

The three method names are strings: `"Automatic"`, `"Fast"`, and `"Galois"`. The default field-degree and relative-group-order limits are 48 and 96. The pair-resolvent degree bound is eight. Disable mathematical size bounds explicitly with

```
"MaxFieldDegree" -> Infinity,
"MaxGroupOrder" -> Infinity
```

and the whole-call time limit with `TimeConstraint -> Infinity`. Removing these limits can cause very long computation and memory exhaustion. A primitive field must be constructed before its degree can be measured, so a degree bound alone cannot bound the cost of that initial operation.

`RadicalSolve` returns an association on success or a `Failure`. `RootToRadicals` extracts the expression on success and propagates a failure otherwise. It never silently returns the input root as if that were a successful radical conversion. To run the supplied tests:

```
TestReport["tests/RadicalRoot.wlt"]
```

12.2 Python API and command line

Install the included Python project in an appropriate environment:

```
python -m pip install ./python
python -m pip install pytest
PYTHONPATH=python python -m pytest tests/test_radicalroot.py -v
```

On Windows, installation makes the `PYTHONPATH` prefix unnecessary. The Python API is

```
import sympy as s
from radicalroot import radicalize, verify_result
x = s.Symbol("x")
r = radicalize(5*x**5 - 25*x**3 + 25*x + 6,
               index=4, method="fast")
assert r.success and verify_result(r)
print(r.expression)
```

Index warning: Python indices are SymPy’s zero-based `CRootOf` indices. Wolfram indices are one-based and their nonreal-root ordering must not be assumed to agree with SymPy’s. The two real examples here use the corresponding shift only because their relevant real-root orders are known. No general index-only conversion bridge is supplied.

The direct Python API has no hard wall-clock timeout. Use the command-line wrapper, which runs the calculation in a separate process:

```
python -m radicalroot --coefficients "5,0,-25,0,25,6" \
  --index 4 --method fast --timeout 30
python -m radicalroot --coefficients "1,0,0,-2" \
  --index 0 --method galois --timeout 120
```

Coefficients are descending rational numbers separated by commas. They are parsed with `fractions.Fraction`, not Python `eval`. The output is JSON. A zero command-line size or time limit disables that limit; in the Python API, `None` disables a degree or group-order bound. A timeout terminates the worker and reports a resource limit, not nonsolvability.

12.3 Result categories

Python status	Meaning
<code>success</code>	Allowed radical expression constructed with exact equality checks; includes method and audit data.
<code>not_solvable</code>	An exact nonsolvable Galois group or nontrivial stable derived subgroup has been computed.
<code>not_found</code>	Bounded fast paths have been exhausted. No conclusion about existence.
<code>resource_limit</code>	A configured degree, order, or worker-time bound was reached. No conclusion about existence.
<code>backend_failure</code>	Required exact CAS operation or internal consistency check did not complete successfully.
<code>unsupported_input</code>	Input or options are outside the stated contract.

Wolfram uses analogous capitalized tags. It also distinguishes a final `VerificationFailure`. For degrees up to six, Python can first use SymPy’s documented Galois-group routine as an exact negative shortcut [7]. A solvable group from that shortcut is not itself treated as a radical expression; construction still proceeds. There is no degree-six restriction on the separately implemented explicit normal-field route.

13 Validation and its limits

The supplied validation records identify the exact software environment and commands. Tests exercise radical grammar rejection, Dickson identities for $n = 2, \dots, 9$, all five quintic conjugates, both real sextic roots, affine power families, factor selection on reducible input, exact pair-resolvent multiplicities, group-chain construction, nonsolvability, invalid input, resource limits, wrong-conjugate rejection, and the notebook degree-eight extension fixture.

The forced general-path fixtures include a quadratic, the real and one nonreal root of $x^3 - 2$, the cyclotomic polynomial $x^4 + x^3 + x^2 + x + 1$, and $x^4 - 2$. Every successful fixture invokes the independent final equality verifier, not just a polynomial-residual test. The finite-group tests include cyclic, dihedral, symmetric and alternating solvable groups, and rejection of A_5 . The polynomial $x^5 - x - 1$ supplies an exact nonsolvable negative test using SymPy's Galois-group computation.

Recorded environment/result	Value
Python	3.13.5
SymPy	1.14.0
pytest	9.0.2
Final Python regression run	48 passed, 0 failed
Aggregate elapsed test time	40.22 seconds
Slowest fixture	General $x^4 - 2$ path, 22.96 seconds
Wolfram runtime tests	Not executed: evaluation service returned HTTP 404

These timings describe this single preparation environment and run. They are neither cross-platform benchmarks nor worst-case bounds. The records are `validation/pytest.xml` (JUnit) and `validation/pytest-output.txt` (console output).

Earlier development runs exposed two ordinary validation defects: an uncaught constant-polynomial construction exception and a test comparing identical polynomials over different declared coefficient domains. Both were corrected before the final recorded test run. A preliminary aggregate run also exceeded its external time budget; the final run, not that interrupted run, is the basis of the reported pass count. This history should not be confused with an all-input performance guarantee.

The Wolfram code has been reviewed and supplied with local tests, but no Wolfram runtime pass count is claimed. Neither implementation has been exhaustively validated over all solvable transitive groups, arbitrary high degrees, or the entire notebook. The current Python backend can spend substantial time simply constructing a primitive element from several high-degree `CRootOf` objects. Failures and resource limits remain first-class outcomes.

14 Cost, optimization, and what is not implemented

The expensive parameter is often the splitting-field degree, not the target's minimal-polynomial degree n . In general $d_L \leq n!$, and

$$D = [L\mathbb{Q}(z_m) : \mathbb{Q}] \leq d_L \varphi(m).$$

This bound does not predict practical running times: coefficient heights, primitive-element choices, algebraic-number membership and expression expansion can dominate. The explicit automorphism table has $|H|^2$ entries. At most D trace candidates are examined per chain step;

each involves exact arithmetic in a D -dimensional rational vector space. The coordinate solves also operate in that ambient space. These are finite but not lightweight computations.

Several improvements are natural but are *not* implemented claims of this release. A Stauduhar-style permutation Galois-group backend can avoid constructing the full splitting field merely to diagnose nonsolvability; such algorithms are developed by Fieker and Klüners [3]. Relative field towers and subgroup-specific invariants could avoid repeated work in a single very large primitive field. Primitive-element reduction and coefficient-height control would make exact arithmetic more practical. A subgroup-chain cost function could prefer small radicands rather than the first available prime quotient. Finally, interval certificates and a serialized quotient-field witness could support a small independent verifier.

A target-specific solver could stop before constructing all of M when the target already lies in a constructed fixed field. The present implementation constructs the full chain for simplicity. It likewise does not minimize the number of radicals, radical nesting depth, algebraic coefficient height, or printed expression size. A shorter answer may require a different extension, a nonobvious change of primitive generator, or a denesting stage after construction. Those optimizations should preserve the exact branch invariants rather than applying unrestricted power identities.

15 Conclusion

The notebook’s extension searches are best understood as useful resolvent heuristics, not a complete criterion for radical solvability. The systematic replacement has a clear mathematical core: compute the normal field, reduce to prime-cyclic fixed-field steps over a sufficient cyclotomic base, construct each radical generator by a finite trace–Fourier search, and carry exact embedding information through every principal-power choice.

This gives an implemented general reference algorithm rather than a degree-four wrapper or an enumeration of increasingly complicated guessed expressions. The structural fast paths solve the motivating examples much more economically. The release deliberately separates three claims: a complete mathematical construction under exact backend contracts, executable bounded reference software, and the finite set of tests actually run. That separation is essential for making the package useful without presenting a timeout or an untested backend as a theorem about radicals.

A Reproduction and source map

Path	Purpose
wolfram/RadicalRoot.wl	Native exact Wolfram implementation.
python/radicalroot/core.py	Structural and Galois–Kummer algorithms.
python/radicalroot/cli.py	Rational-input CLI with worker timeout.
tests/test_radicalroot.py	Executed Python regression tests.
tests/RadicalRoot.wlt	Wolfram tests for local execution.
examples/basic.py, basic.wl	Entry-point examples.
validation/	Actual test logs and status notes.
provenance/	Notebook extraction and input hashes.
tools/extract_notebook.py	Non-evaluating inspection extractor.

Compile the article by running `make pdf` or two passes of `pdflatex radicalroot.tex` in `article/`. The small `validation_summary.tex` file is part of the article source and must remain beside it.

No font files, external images, or external bibliography databases are needed. The original input archive is identified by hash but is not repackaged as newly licensed source.

B A schematic algorithm

```

RADICALIZE(alpha):
  f := minimal polynomial of the selected embedded alpha
  try structural radical candidates, accepting exact equality only
  L := the exact splitting field of f in the fixed complex embedding
  m := product of distinct primes dividing [L:Q]
  M := compositum(L, Q(zeta_m)); choose an integral primitive theta
  A := all exact automorphisms of M, by splitting minpoly(theta) in M
  H := automorphisms in A fixing zeta_m
  if H has a nontrivial stable derived subgroup:
    return NOT_SOLVABLE
  chain := prime-cyclic subnormal chain of H
  B := rational power basis of Q(zeta_m)
  expressions := matching symbolic basis
  for S > N in chain, p = [S:N], sigma outside N:
    find first nonzero rho among the D trace-Fourier basis candidates
    verify N-fixedness, sigma(rho) = zeta_p*rho, S-fixedness of rho^p
    express rho^p in B by exact rational linear algebra
    determine rho / principal_root(rho^p,p) exactly in {zeta_p^b}
    define the corresponding branch-correct radical
    enlarge B and its expression basis by rho^j, 0 <= j < p
  express alpha in the final basis and substitute radical definitions
  verify allowed syntax and exact embedded value
  return expression and audit data

```

The actual wrappers surround this scheme with explicit input checks and resource limits. No “try until lucky” step remains in the complete mathematical core.

References

- [1] V. Reshetnikov, *Why is ToRadicals not able to handle all cases? Is there a workaround?* Mathematica Stack Exchange, question 34011, including its discussion and answers. <https://mathematica.stackexchange.com/q/34011>. Accessed 8 September 2026. The two motivating polynomials are from this question; worked derivations here are independently presented.
- [2] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022. In particular, independence of homomorphisms, Proposition 5.27, and Theorem 5.34. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [3] C. Fieker and J. Klüners, *Computation of Galois groups of rational polynomials*, LMS Journal of Computation and Mathematics **17**(1) (2014), 141–158. Preprint <https://arxiv.org/abs/1211.3588>.
- [4] Wolfram Research, *ToRadicals*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/ToRadicals.html>. Accessed 8 September 2026.

- [5] Wolfram Research, *ToNumberField*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/ToNumberField.html>. Accessed 8 September 2026.
- [6] Wolfram Research, *RootReduce*, *FactorList*, and *AlgebraicNumberPolynomial*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>; <https://reference.wolfram.com/language/ref/FactorList.html>; <https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html>. Accessed 8 September 2026.
- [7] SymPy developers, *Number Fields*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/numberfields.html>. The documented Galois-group shortcut has degree bound six; the number-field operations support the separate general construction in this release. Accessed 8 September 2026.
- [8] SymPy developers, *Domains Reference*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/domainsref.html>. Algebraic fields and exact quotient-field elements. Accessed 8 September 2026.